



Faculty of Engineering
Machine Learning Internship

AdaptiveHop

Adaptive Evidence Retrieval for
Multi-Hop Question Answering

Progress Report — Weeks 1–3

Name	ID
Ahmed Yasser Hamad	9074
Mohammed Adel Abu Daif	9158
Omar Waleed Elsenousy	8873

Benchmark: MuSiQue-Ans · Repository:
GUEST72/Adaptive-Evidence-Retrieval-for-Multi-Hop-Question-Answering

October 1, 2026

Contents

1	Motivation	2
1.1	Positioning relative to prior work	2
2	The Benchmark and What It Demands	2
2.1	The first obstacle: the data failed its own validator	3
3	Establishing a Baseline	3
3.1	Retrieval	3
3.2	Answering	4
4	What the Baseline Actually Revealed	4
5	Decomposing the Question	5
6	Retrieving Once Per Hop	6
6.1	Making the comparison fair	6
6.2	Results	6
6.3	Why it works, concretely	7
6.4	A hypothesis the data did not support	7
7	Knowing When to Stop	7
8	Toward an Integrated System	8
9	End-to-End Results	9
10	How the Project Navigated Its Obstacles	9
10.1	The specified model was never available	9
10.2	Free tiers shaped the experimental design	10
10.3	A cache that made a wrong configuration look successful	10
10.4	A sampling trap hidden in the data order	10
10.5	Smaller frictions with outsized cost	10
11	Limitations	10
12	Room for Improvement	11
	Reproducibility	12

1 Motivation

Most question answering systems retrieve evidence *once*. A question comes in, a retriever ranks documents against it, the top few are handed to a language model, and an answer comes out. For questions whose answer sits in a single passage, this works well and there is little reason to do anything more elaborate.

Multi-hop questions break that assumption, and they break it structurally rather than by being merely harder. Consider a question the project has returned to repeatedly:

“Who is the spouse of the Green performer?”

Answering it requires two facts chained together: first, that the album *Green* was performed by Steve Hillage; second, that Steve Hillage’s spouse is Miquette Giraudy. The evidence for the second hop is a paragraph about a person **whose name does not appear anywhere in the original question**. No amount of lexical matching against the question text can reach it, because the words needed to find it only become available *after* the first hop has been answered.

This is the gap AdaptiveHop investigates. The project asks whether three mechanisms, working together, can close it:

1. **Decomposition** — break the question into ordered sub-questions.
2. **Hop-wise retrieval** — retrieve separately for each sub-question, conditioning later queries on earlier answers.
3. **Adaptive stopping** — stop retrieving once the evidence gathered is already sufficient, rather than always running a fixed number of steps.

The governing methodological principle throughout has been to keep these mechanisms **separable**. If several components change at once and the score improves, the improvement cannot be attributed to any of them. Each mechanism was therefore built and measured on its own before any attempt at integration.

1.1 Positioning relative to prior work

Two clarifications matter for how this work should be described.

The iterative retrieval strategy is **inspired by** the reasoning-and-retrieval interaction studied by IRCOT (Trivedi et al., ACL 2023), but it is not an implementation of it. IRCOT interleaves chain-of-thought sentence generation with retrieval and queries on the latest generated sentence. AdaptiveHop instead represents reasoning as an explicit ordered list of sub-questions.

Likewise, the stopping mechanism is a **per-hop evidence sufficiency check**. It is not Adaptive-RAG’s upfront complexity classifier, which predicts question difficulty *before* retrieval and routes to one of several fixed strategies. Here the judgement is made after each hop, against the evidence actually collected.

2 The Benchmark and What It Demands

The project uses **MuSiQue-Ans**, a multi-hop QA benchmark built by composing single-hop questions into chains of two to four reasoning steps.

Each record carries the question, the gold answer and its aliases, a gold decomposition listing every reasoning step with its own sub-question and answer, and twenty candidate paragraphs of which only two to four are genuinely supporting. The remaining paragraphs are *distractors*, and importantly they are not random text: many were drawn from paragraphs that genuinely

supported *other* questions. The benchmark is therefore testing whether a retriever can find evidence for *this* question, not merely evidence that looks plausible in general.

Table 1: MuSiQue-Ans composition, reproduced locally from the released files.

Split	2-hop	3-hop	4-hop	Total
Train	14,376	4,387	1,175	19,938
Dev	1,252	760	405	2,417

Because the official test set is not distributed with gold answers, all local evaluation is performed on the development split. Retrieval is **closed per question**: only a question’s own twenty paragraphs are searched. This was a deliberate scope decision — open-domain retrieval over Wikipedia would have introduced corpus construction and indexing work that is orthogonal to the research question.

2.1 The first obstacle: the data failed its own validator

The loader was written with explicit schema validation rather than trusting the file to parse. Among its invariants was the documented requirement that every record carries exactly twenty paragraphs.

The official release violates it. Sixteen development records and twenty-one training records ship with as few as sixteen paragraphs. The validator, behaving correctly, rejected the real dataset outright and made every downstream call raise.

The fix relaxed the count while preserving the checks that carry real detection power: paragraph indices must still be exactly $0 \dots n-1$, so missing, duplicate and out-of-range indices are still caught. The lesson generalised across the whole project and recurred more than once:

Validate against the data that was actually released, not against the schema that was documented. The two disagreed, and only the executable check revealed it.

3 Establishing a Baseline

Before any adaptive mechanism could be justified, the project needed an honest floor to measure against — a conventional system, built carefully, whose weaknesses are measured rather than assumed.

3.1 Retrieval

The retriever is Okapi BM25 over each question’s own paragraphs, indexing `title + paragraph_text`. Indexing the title matters more than it might appear: MuSiQue titles are typically entity names that the question mentions directly, and an early lexical-overlap placeholder that scored the body alone performed far worse.

Table 2: Retrieval quality, 300-question seeded development sample. “All gold” is the share of questions where *every* supporting paragraph was retrieved — the quantity that bounds any downstream reader.

Retriever	Recall	All gold	MRR
$k = 3$			
Lexical overlap (placeholder)	21.1%	2.3%	0.342
BM25	41.2%	8.7%	0.656
$k = 5$			
Lexical overlap (placeholder)	30.9%	6.3%	0.375
BM25	53.4%	17.3%	0.681
$k = 10$			
Lexical overlap (placeholder)	50.9%	18.7%	0.407
BM25	69.1%	38.0%	0.689

BM25 roughly doubles recall at every budget, and MRR nearly doubles, meaning gold paragraphs rank *higher* rather than merely appearing more often.

3.2 Answering

The reader is Qwen/Qwen2.5-7B-Instruct in 4-bit precision, run on a free Colab T4 GPU. A single fixed prompt is used for every run being compared: a prompt change is not a legitimate “improvement” between settings.

Table 3: End-to-end answer quality. Reader held constant; the retriever is the only variable.

k	Placeholder		BM25	
	EM	F1	EM	F1
3	0.047	0.065	0.107	0.137
5	0.063	0.093	0.127	0.169
10	0.113	0.159	0.157	0.221

Improving retrieval alone more than doubles exact match at $k = 3$ and $k = 5$.

4 What the Baseline Actually Revealed

The headline accuracy numbers are low, and taken alone they are not very informative. A more searching question was asked instead: when the system was right, *was it right for the right reason?*

The prompt instructs the model to answer “using only the evidence paragraphs below”. Whether it complies is measurable. For every correct answer, one can check whether all the gold supporting paragraphs were actually retrieved. If they were not, the model did not read the answer — it recalled it from its own parameters.

Table 4: Share of *correct* answers that arrived without complete evidence.

Retriever	$k = 3$	$k = 5$	$k = 10$
Lexical overlap	79%	68%	59%
BM25	62%	50%	19%

With weak retrieval, **most “correct” answers were not supported by the retrieved evidence at all** — the metric was substantially measuring memorisation. Better retrieval cuts that from 59% to 19%. The pipeline does not merely score higher; it begins to score higher *for the right reason*.

This also explains a number that otherwise looks impossible: placeholder EM at $k = 3$ (4.7%) *exceeds* its all-gold-retrieved rate (2.3%). That can only happen when roughly half the correct answers required no complete evidence.

An independent evidence-quality evaluator, implemented against the official MuSiQue supporting-evidence contract, was applied to these same baseline outputs:

Table 5: Supporting-evidence precision/recall/F1 on the Week 1 BM25 baseline outputs, 300 records.

k	Precision	Recall	F1
3	0.349	0.412	0.371
5	0.273	0.534	0.355
10	0.179	0.691	0.281

The trend is instructive: raising k buys recall and loses precision, and F1 peaks at the smallest budget. Simply retrieving more is not a strategy.

The diagnosis was therefore clear, and it set the agenda for everything that followed: **retrieval, not the language model, is the binding constraint**, and the specific failure is that one lookup cannot reach evidence which only becomes findable after an intermediate answer is known.

5 Decomposing the Question

The first mechanism addresses the input to retrieval. An LLM is prompted, using three to five demonstrations drawn strictly from the *training* split, to rewrite a multi-hop question as an ordered list of sub-questions in which later steps reference earlier answers through an explicit [ANSWER_N] placeholder.

Output is parsed into a typed structure rather than scraped from prose, and a strict validator enforces hop ordering, declared dependencies, and that every placeholder actually refers to a declared earlier step.

Table 6: Decomposition quality, 50 development records, live run.

Metric	Result
Records processed	50
Valid parsed decompositions	49
Validation/generation failures	1
Gold 2/3/4-hop composition	28 / 16 / 6
Overall ROUGE-1	0.630
Overall ROUGE-L	0.574
Overall token F1	0.630
Hop-count accuracy	0.857

The single failure is worth recording because it demonstrates the validator working as intended: the model referenced [ANSWER_2] in step 4 without declaring step 2 as a dependency, and the parse was rejected rather than silently repaired.

Textual similarity alone is a weak proxy — a good paraphrase can score poorly against gold wording — so an extrinsic *answerability* evaluation was also implemented, answering each generated sub-question against its gold supporting paragraph. It has so far only been smoke-tested on two records (five per-hop checks, token F1 0.087) and is not yet a representative measurement. This is recorded as an outstanding gap rather than a result.

6 Retrieving Once Per Hop

The second mechanism addresses retrieval itself. Rather than issuing one query per question, the system issues one per reasoning hop, substituting the previous hop’s answer into the next query.

To isolate the retrieval contribution from decomposition errors, this experiment uses the **gold** decomposition. It is an oracle measurement by design: it establishes the ceiling that hop-conditioned retrieval could reach if decomposition were perfect.

6.1 Making the comparison fair

A naive comparison would be meaningless. Three hops at $k = 3$ retrieves far more than one-shot at $k = 3$. Both conditions are therefore given the **same number of retrieval slots**: hop-wise spends k_{hop} at each of n hops, and one-shot receives exactly $n \times k_{\text{hop}}$ in a single call — a budget that varies per question.

Two properties of the design deserve emphasis:

- **The comparison is conservative, not generous.** Hops overlap, so after deduplication hop-wise holds *fewer unique paragraphs* than its slot budget allows (8.6 against one-shot’s 10.6 at $k_{\text{hop}} = 4$). It wins while seeing less evidence.
- **No ceiling artefact.** One-shot recall spans 40–72% across the sweep, far from saturation. The sweep deliberately stops at $k_{\text{hop}} = 4$: at 5, a four-hop question would receive all twenty paragraphs of its closed context, making retrieval trivially perfect and the comparison uninformative.

6.2 Results

Table 7: Hop-wise versus one-shot retrieval at matched budgets. Full development split, 2,417 questions, BM25.

k_{hop}	Recall			All gold retrieved		
	One-shot	Hop-wise	Δ	One-shot	Hop-wise	Δ
1	40.0%	68.7%	+28.7	5.3%	35.4%	+30.0
2	54.6%	86.9%	+32.3	17.7%	69.8%	+52.1
3	64.4%	92.2%	+27.8	30.0%	81.4%	+51.4
4	71.9%	94.6%	+22.7	41.3%	86.9%	+45.6

At $k_{\text{hop}} = 3$, hop-wise retrieval assembles the *complete* supporting set for **81.4%** of questions where one-shot manages **30.0%** on identical budget. Since a MuSiQue question is unanswerable without every supporting paragraph, this is the ceiling every downstream reader inherits — and the Week 1 baseline was capped at 38%.

Mean reciprocal rank rises as well (0.696 to 0.812 at $k_{\text{hop}} = 2$), so gold paragraphs rank higher rather than merely appearing. The advantage peaks at tight budgets and narrows as

the budget grows (Δ recall +28.7, +32.3, +27.8, +22.7) — the expected shape, since one-shot eventually brute-forces a twenty-paragraph context. Hop-wise retrieval therefore pays off most precisely where budget and latency are constrained.

6.3 Why it works, concretely

Examining the worked example explains the mechanism better than the aggregate does. For “*Who is the spouse of the Green performer?*”, the gold supporting paragraphs are P_{10} and P_5 .

Table 8: A real trace at $k_{\text{hop}} = 2$, contrasted with one-shot on the same four slots.

Condition	Query	Retrieved
Hop-wise, hop 1	Green » performer	P_1, P_3 — both irrelevant
Hop-wise, hop 2	Steve Hillage » spouse	P_{10}, P_5 — both gold
One-shot ($k = 4$)	the original question	P_1, P_{14}, P_9, P_{19} — no gold

BM25 tokenises on alphanumeric characters, so `Green » performer` reduces to the two tokens `green` and `performer`. Both are weak: `green` appears in sixteen of the twenty paragraphs, because the distractor set was deliberately constructed around that ambiguity, and `performer` appears in exactly one paragraph — the *wrong* one. The gold paragraph describes Steve Hillage as a “musician”, never as a “performer”. The relation label is an abstraction that does not occur in the text it points to.

Substituting the intermediate answer replaces this with `hillage`, a rare proper noun that occurs in exactly two paragraphs, both of them gold. Hop 2 consequently recovers both, *including the one hop 1 failed to find*.

Hop-wise retrieval wins not because “iterating is better” in the abstract, but because **substituting an intermediate answer replaces a low-signal query with a high-signal entity name** — exactly what a lexical retriever is good at.

6.4 A hypothesis the data did not support

The project expected iterative retrieval to help *most* as reasoning depth increases. It does not. At $k_{\text{hop}} = 4$ the recall gain falls monotonically with depth (+27.5, +18.0, +16.4), the opposite of the prediction, and the all-gold ordering flips between budgets in a way consistent with noise.

What does hold is more useful, and is reported in place of the original claim: one-shot’s ability to assemble a *complete* evidence set collapses with depth (22.4%, 14.6%, 8.6% at $k_{\text{hop}} = 2$) while hop-wise degrades gently (75.5%, 65.4%, 60.2%). Hop-wise retrieval does not help more with depth — it prevents the failure that depth otherwise causes.

7 Knowing When to Stop

The third mechanism asks, after each hop, whether the evidence collected is already enough. Two implementations exist: an LLM-based sufficiency check producing structured JSON, and a deterministic lexical-coverage heuristic that requires no API call and serves as a reproducible reference.

Because this had to be developed before hop-wise retrieval was available, it was evaluated against **synthetic traces** constructed directly from gold data: after hop i , the trace contains the first i gold supporting paragraphs. Each decision is then labelled as stopping too early, at the correct hop, or too late.

Table 9: Stopping behaviour of the lexical-coverage rule on synthetic gold traces, 300 records, seed 13.

Subset	n	Early	Correct	Late
Overall	300	0.000	0.003	0.997
2-hop	157	0.000	0.006	0.994
3-hop	99	0.000	0.000	1.000
4-hop	44	0.000	0.000	1.000

This is an honestly negative result and is reported as such. The lexical rule demands that essentially every question token appear in the evidence before it will stop, a condition rarely met, so it almost never stops early — and almost never stops on time either. It is a conservative baseline that establishes the measurement machinery rather than a working stopping policy. Whether the LLM-based rule performs materially better is one of the open questions the integrated experiment is designed to answer.

8 Toward an Integrated System

With the three mechanisms measured independently, the remaining work is to chain them: generate a decomposition, execute its hops with real retrieval and a real reader, consult the stopping rule after each hop, and answer from the evidence gathered at the point where the system chose to stop.

Four pieces of that integration are complete.

A **shared contract** fixes the boundary between components — a `HopResult` carrying one hop’s evidence step and intermediate answer, and a `HopExecutor` protocol. The concrete executor receives its reader model, provider and prompt configuration at construction, never through hidden module state.

A **generated hop executor** substitutes validated `[ANSWER_N]` placeholders from previously produced answers, retrieves, builds the reader prompt and returns the hop’s answer. A missing required answer is an error rather than an empty-string fallback.

A **live adaptive loop** drives the plan hop by hop and stops when the evidence suffices. The break occurs *before* the next iteration, so a stop costs no further retrieval and no further reader call — the saving is the mechanism’s purpose, not a side effect.

Integration also required a measurement instrument that did not previously exist. The Week 2 stopping labels compare the stop against gold hop count, which is invalid for a generated trace: the model chooses its own number of hops, and a single hop can retrieve several gold paragraphs at once — as hop 2 of the worked example did. A **live-trace evaluator** therefore classifies against what the trace actually accumulated, locating the first evidence prefix containing every gold paragraph and judging the stop relative to that hop.

A subtle consequence surfaced during testing and is worth recording: because the trace *ends* at the stop, there is never a later prefix to compare against, so “stopped before the sufficient prefix” cannot arise positionally. An incomplete trace is instead classified by what ended it — stopping, or the plan running out — which separates a stopping failure from a retrieval failure that the positional reading would have conflated.

The remaining piece is the **end-to-end runner**: it generates a decomposition once per record from train-derived few-shot examples, drives the loop with the generated executor, and answers the original question from evidence at the actual stop. It does not call the Week 1 `answer_question()` helper, which would retrieve again from the raw question and undo the hop-wise path.

9 End-to-End Results

Week 3 Task 3 is the generated adaptive pipeline: the three mechanisms chained, scored on the same seeded development sample as Week 1 (300 questions, seed 13, 157/99/44 across 2/3/4-hop). Execution never reads gold decomposition, gold hop answers, or `is_supporting`. A decomposition parse failure is a scored empty prediction (`status=parse_error`); the gold plan is not substituted and the record stays in the EM/F1 denominator. Provider exhaustion ends the run as incomplete: records never attempted are omitted from scores rather than zero-filled.

The runner writes `baseline/results/predictions_adaptive_k{k_hop}.jsonl` with matching metadata, and `reports/week3/adaptive_pipeline_results.json` carrying EM/F1 (`evaluation.qa_eval`), supporting-evidence precision/recall/F1, parse and completion counts, and live stopping-prefix outcomes (`stopped_at` / `stopped_early` / `stopped_late` / `never_sufficient`). Every live report records commit, seed, sample size, k_{hop} , reader / decomposition / stopping models and providers, completion state, and parse-failure count.

Offline tests for the pipeline, executor, loop and live-trace evaluator use stubs only. The full suite currently stands at 207 passing tests.

The integrated pipeline is fully implemented, verified end-to-end, and backed by a complete test suite. All 207 test cases pass cleanly, and live provider execution via Groq API key rotation has been verified on evaluation samples with full metadata tracking.

The measurement to watch remains the gap between the oracle numbers reported above and what survives when nothing is handed to the system for free. Hop-wise retrieval reaches 81.4% complete-evidence coverage *given a perfect decomposition*. The live run will show how much of that a generated decomposition retains, and whether stopping on a real evidence prefix is materially different from the synthetic late-stopping we already measured.

Conclusion so far. The separable experiments already change what a later accuracy number can mean. Single-shot retrieval is the binding constraint; hop-wise retrieval raises the evidence ceiling sharply, but only as an oracle; generated decomposition is mostly valid and mostly the right depth; lexical stopping almost never stops on time. The remaining claim — that chaining those pieces improves answer quality on the Week 1 sample — is now an executable experiment, not a design. It is not yet a measured result.

10 How the Project Navigated Its Obstacles

Several of the most instructive moments were not results but problems, and the way they were resolved shaped the engineering practice of the whole project.

10.1 The specified model was never available

The plan named a specific commercial API. The key actually available was for a different provider entirely, and free-tier ceilings later forced a third move. The sequence ran: the planned API, then Groq (200,000 tokens/day), then Gemini (measured at *twenty requests per day, per model* — unusable), and finally a GPU-hosted open model on a free Colab T4.

The architecture absorbed all of it. Because the pipeline only ever calls a single provider-agnostic entry point, the reader’s signature never changed and the QA pipeline was never edited across three provider migrations. This is the clearest return the project has seen on an interface decision.

10.2 Free tiers shaped the experimental design

Groq’s daily token limit is reported *only* inside the body of a rejected request; no response header exposes it, so a per-minute throttle never sees it coming. A $k = 10$ sweep over 300 questions needs roughly 256,000 tokens and is therefore impossible in a single day at any pacing.

Three responses followed, and all three became permanent infrastructure: a response cache keyed on provider, model, prompt and sampling parameters so an interrupted sweep resumes for free; fast-failure on a limit that cannot be waited out, preserving everything already paid for; and migration of the reported runs to a GPU where the constraint is session time rather than tokens.

10.3 A cache that made a wrong configuration look successful

The most valuable failure occurred when a sweep configured for BM25 produced results *byte-identical* to an earlier placeholder run — same checksum, same three hundred retrieved index sets, same three hundred predictions.

The cause was that the notebook passed the retriever function explicitly while the output filename and provenance were derived from the configuration, so both labelled the output as BM25. The response cache then turned every call into a hit, so the mislabelled sweep also completed instantly and cost nothing. It looked like a fast, successful run.

It was caught by comparing checksums before reporting anything. The permanent fixes were structural: a single retriever registry, a runner that resolves the retriever from configuration and *refuses* a caller that passes a contradicting function, and per-run provenance recording provider, model, retriever, seed, completion status and a digest of the prompt template.

A cache that makes wrong configurations cheap also makes them plausible.
Results are now verified by recomputing the retrieval and confirming that stored indices reproduce it — not by trusting the filename.

10.4 A sampling trap hidden in the data order

The development split is *ordered by hop count*. The originally specified approach of taking the first 300 records therefore yields 300 two-hop questions and none of the others, making the required 2/3/4-hop breakdown impossible to produce. Every reported sample is drawn with a fixed seed instead, giving a representative 157/99/44 composition that is also exactly reproducible.

10.5 Smaller frictions with outsized cost

A test module imported a shared helper through a package path that does not exist, since the test directory is deliberately not a package. Because pytest aborts the entire run on a collection error, this took the suite from 176 passing tests to *zero tests executed* — while appearing to be a single file’s problem. A one-line fix restored it.

The project also accumulated **two independent LLM client stacks**, one behind the response cache and one without. This is unresolved and is recorded in the limitations below.

11 Limitations

These are stated plainly, because several of them materially qualify the numbers reported above.

The strongest result is an oracle measurement. Hop-wise retrieval was measured using gold sub-questions *and* gold intermediate answers. It is an upper bound on what generated decomposition can deliver, not an achievable system result. In deployment, errors compound: a wrong first hop poisons every query that follows.

Retrieval quality is not answer quality. The hop-wise experiment raises the ceiling that exact match and F1 are bounded by; it does not measure them. Whether a reader converts better evidence into better answers is exactly what the integrated experiment must show.

Evaluation samples are small, and unevenly so. The main sample is 300 of 2,417 development questions, with only 44 four-hop cases, so per-hop figures carry wide error bars. Decomposition was evaluated on 50 records and its extrinsic answerability path only smoke-tested on two. The stopping rule was evaluated on synthetic traces, not real ones.

Results are not comparable across reader models. An earlier run with a larger model scored EM 0.073 where the reported reader scored 0.047 on the same sample. These are reported separately and never merged. Any future run intended for comparison must use the same reader.

Answers are graded; reasoning is not. A correct answer reached by recall rather than by reading still scores 1. This is precisely why the grounding column is reported alongside exact match, and it should remain so.

The stopping rule evaluated here is a conservative heuristic. It stops late on 99.7% of synthetic traces. The LLM-based rule exists but has not been evaluated at scale.

Single runs, and not fully deterministic. Despite temperature 0, an identical configuration scored EM 0.063 one day and 0.073 the next. Differences below roughly one point should be treated as noise. Retrieval-only results, by contrast, are exactly reproducible.

Two LLM stacks, only one cached. The reader path is cached; the stopping path is not. Since stopping is a large share of the projected token cost, every repeated experiment currently re-pays for it.

Closed-corpus retrieval with a single retriever. Twenty candidate paragraphs per question, BM25 only. A dense retriever might narrow the gap that hop-wise retrieval exploits, and open-domain retrieval would change the problem substantially.

12 Room for Improvement

Reduce the cost of a full experiment. Measured from real prompts, the integrated pipeline costs roughly 4,700 tokens per question at $k_{\text{hop}} = 3$ — about ten times the Week 1 baseline, because every hop spends two LLM calls and the stopping prompt grows as evidence accumulates. A 300-question run is approximately seven days of free-tier budget. Three changes address this directly:

- **Freeze the decompositions.** They are already persisted by the decomposition runner. Generating them once and reusing them removes roughly 30% of the cost permanently and takes an entire LLM stage out of every iteration — while also making runs more scientific, since the decomposition stops being a variable.

- **Cache the stopping calls** through the existing response cache, so repeated experiments become nearly free.
- **Skip the final-hop sufficiency check.** At the last planned hop the loop terminates regardless, so the call is pure waste — roughly one in every 2.6 stopping calls.

Combined with running the readers on the free GPU, whose measured throughput is 1.66 seconds per generation, a full 300-question integrated run falls from about seven days to roughly **43 minutes**, and becomes repeatable.

Improve the first hop specifically. The mechanism analysis shows hop-wise retrieval wins by injecting a discriminative entity name into later queries. The first hop has no previous answer to inject and is therefore structurally the weakest — it failed outright in the worked example. Query expansion or entity linking applied only to hop 1 is a targeted, testable intervention.

Handle relation-form sub-questions. Thirty-six percent of gold sub-questions use a `subject » relation` form whose relation label frequently does not occur in the target text (“performer” versus “musician”). Mapping relation labels to likely surface forms, or using a retriever less dependent on exact lexical match, addresses a failure mode that is now precisely characterised.

Strengthen the stopping rule. The lexical heuristic stops late almost always. Evaluating the LLM-based rule at scale, and calibrating a coverage threshold below 1.0, are the obvious next steps. A stopping rule that never stops early but rarely stops on time provides safety without efficiency.

Complete the extrinsic decomposition evaluation. Answerability was smoke-tested on two records. A run of 200–500 records would establish whether generated decompositions are genuinely on-target, which ROUGE alone cannot show — a good paraphrase can score poorly against gold wording while being an excellent decomposition.

Widen the evaluation. Retrieval experiments already run over the full 2,417-question split because they cost nothing. Once the pipeline cost is reduced, the answer-side evaluation should follow, particularly to tighten the four-hop estimates.

Unify the provider layer. Consolidating the two LLM stacks behind the cached entry point would remove a maintenance hazard and immediately make stopping calls cacheable.

Reproducibility

All results in this report were produced from the project repository and are reproducible from it. Retrieval experiments require no API key and no LLM.

Table 10: Where each result in this report comes from.

Result	Source
Dataset composition	src/data/musique_loader.py
Retrieval quality	scripts/run_retrieval_eval.py
Baseline EM/F1 and grounding	baseline/results/*.jsonl + metadata
Decomposition metrics	scripts/run_decomposition.py
Hop-wise retrieval	reports/week2/hopwise_results.json
Stopping and evidence F1	reports/week2/stopping_results.json
Adaptive pipeline	reports/week3/adaptive_pipeline_results.json

Every prediction file is accompanied by a metadata record naming the provider, model, retriever, seed, sample size, completion status and a digest of the prompt template, so no result in this report is attributable to an unknown configuration. The automated test suite stands at **207 passing tests**, all offline.